

Fase 1 — Migration de identidade de canal · status de entrega

Por que este arquivo não tem número de bloco. Conferido no ClickUp em 2026-09-21: a trilha da automação é a lista "OCR Contratos (WhatsApp) — Breno", que tem numeração própria de 1 a 8 e não usa blocos `m.n`. Esta rodada cai sob a tarefa 4. Tratar os casos que dão errado, em andamento. Os blocos `m.n` são da lista "Plataforma Gestão de Obras", outra trilha — usar um número de lá aqui seria o que o `CLAUDE.md` proíbe. Nome descritivo, então, e a pasta é `docs/automacao/`.

Data: 2026-09-21 · Branch: `feature-dev-breno-automacao` · Ambiente: `gc-dev` (`gzbmhgnpoehornmidmgg`) · Plano: `docs/automacao/migracao-telegram-integracao-automacao.md` , Fase 1.

Resumo

Fase 1 concluída e aplicada em `gc-dev`. As sete camadas do `npm run validar` passam, com todos os números iguais à baseline de 2026-09-10 — que é o resultado correto, porque a Fase 1 é schema e documentação, não acrescenta rota, helper nem Server Action. Uma pendência nominal permanece: a decisão 6 (`PROPOSTA_REGISTRADA`) depende do `n8n`.

1. O que foi aplicado

1.1 `supabase/migrations/20260921150000_contatos_canal.sql`

Aplicada por `supabase db push`. `migration list` agora mostra 14/14 alinhadas entre local e remoto (eram 13/13 antes).

Conferido por query no catálogo, depois de aplicar:

CHECAGEM	RESULTADO REAL
Colunas novas	<code>contatos_whatsapp.canal</code> (text, NOT NULL), <code>.telegram_chat_id</code> (text, null), <code>.telefone</code> agora nullable ; <code>documentos_processamento.canal</code> , <code>.canal_chat_id</code> , <code>.proposta_criada_id</code>
Unique global de telefone	removido — <code>contatos_whatsapp_telefone_key</code> não existe mais
Uniques parciais por canal	<code>idx_contatos_telefone_whatsapp</code> , <code>idx_contatos_chat_id_telegram</code>
FK de obra	<code>contatos_whatsapp_obra_fk</code> → FOREIGN KEY (obra_id, empresa_id) REFERENCES obras(id, empresa_id) ON DELETE RESTRICT — composta , era simples

CHECAGEM	RESULTADO REAL
FK da proposta	<code>documentos_processamento_proposta_criada_fk</code> → <code>(proposta_criada_id, empresa_id)</code> <code>REFERENCES propostas(id, empresa_id) ON DELETE SET NULL (proposta_criada_id)</code>
CHECKS	<code>contatos_whatsapp_canal_check</code> , <code>contatos_whatsapp_identidade_do_canal</code> , <code>documentos_processamento_canal_check</code>

Os nomes auto-gerados que a migration derruba estavam certos. Com o banco no ar deu para conferir antes de aplicar: `contatos_whatsapp_telefone_key` e `contatos_whatsapp_obra_id_fkey` existiam com exatamente esses nomes. O bloco de verificação de 5 checagens não precisou disparar, mas continua no arquivo como rede para quem reaplicar noutro ambiente.

1.2 Backfill — 29/29 classificados, mas só 8 correlacionáveis

TABELA	LINHAS	RESULTADO
<code>contatos_whatsapp</code>	2	2 WHATSAPP, 0 TELEGRAM, 0 sem telefone
<code>documentos_processamento</code>	29	29 WHATSAPP, 0 sem canal, 8 com <code>canal_chat_id</code>

Achado que vale registrar: o backfill classificou os 29 documentos, mas só 8 ficaram com `canal_chat_id` preenchido. Os outros 21 não têm `dados_extraidos->'telefone'` — nunca tiveram. Não é defeito da migration: é a prova, em dado real, de que a correlação resposta↔documento por dentro do JSON já era frágil antes da troca de canal. Para esses 21 a resolução manual de revisão nunca teria casado, nem pelo caminho velho. Não há o que recuperar; fica como contexto para a Fase 4.

1.3 Profile de serviço (decisão 12)

`bash scripts/aplicar-seed.sh supabase/criar_usuario_automacao.sql`, verificado em seguida:

```

uuid      = 35d00e89-8c84-4135-8601-f1ad8494e659
nome      = Automação (Telegram)
email     = automacao@obraminds.com
perfil    = visualizador
empresa   = LC EMPRESA
ativo     = True
pode_logar = False      <- sem linha em auth.identities, como desenhado

```

O uuid foi gravado em `.env.local` como `INGESTAO_PROFILE_ID`, que é de onde a Fase 6 vai lê-lo. (`.env.local` não é versionado.)

1.4 npm run db:types:dev

src/lib/supabase/types.ts regenerado: 44 linhas acrescentadas, 15 removidas. As seis colunas novas aparecem nas três variantes (Row , Insert , Update) das duas tabelas — 11 ocorrências de telegram_chat_id / canal_chat_id / proposta_criada_id .

1.5 Contradição do migration repair — resolvida

Item explícito da Fase 1. CLAUDE.md e supabase/README.md diziam que o repair das dez versões era pendência aberta, sem dizer que é de prod; quem lesse o CLAUDE.md primeiro travava sem motivo. Confirmado na prática hoje: o db push rodou em gc-dev sem nenhum preparo.

1.6 n8n-mcp no escopo do gc-sistema

A decisão 13 dizia pôr no .mcp.json do repositório. Não foi feito assim, de propósito: a configuração carrega um N8N_API_KEY de 267 caracteres, e .mcp.json na raiz é versionado — seria commitar uma chave de API, num repo onde senha em texto claro já vazou uma vez (registrado em supabase/criar_usuario_admin.sql , 2026-09-05).

Foi para ~/.claude.json , escopo do projeto /Users/a1234/gc-sistema , que é onde o keen-mendel guarda a mesma configuração. Diff estrutural contra backup: uma única chave acrescentada. Só carrega na próxima sessão.

2. Números reais do npm run validar

Rodado em 2026-09-21 contra gc-dev, comparado com a baseline de 2026-09-10 (Seção 0.7 do plano).

CAMADA	BASELINE	AGORA	
estático	tsc limpo, lint sem warnings	tsc --noEmit limpo, No ESLint warnings or errors (82s)	ok
unitário	51 casos, 0 falhas	51 casos, 51 pass, 0 falhas (4s)	ok
build	31 rotas	31 rotas, iguais à baseline (49s)	ok
runtime	41/41 rotas	41/41 rotas ok como breno@obraminds.com (19s)	ok
dados	12/12 checagens	12/12 checagens ok sob RLS (3s)	ok
escrita	38/38 passos	38/38 passos ok, nada sobrou em gc-dev (16s)	ok
navegador	24/24 passos, 13 screenshots	24/24 passos ok, nenhum erro de console (182s)	ok

Todos os sete números batem a baseline. É o resultado esperado e é o que prova a ausência de regressão: a Fase 1 mexe em schema, tipos e documentação, e não acrescenta rota, helper nem Server Action. Rota nova (/api/ingestao/proposta , baseline indo de 31 para 32) é da Fase 6.

Um flake registrado, porque aconteceu: na primeira execução do plano inteiro, a camada navegador reprovou em `esperando demais por: /propostas/nova` (2/3 passos). Repetida isoladamente, passou 24/24. A causa é compilação a frio do `next dev` na primeira navegação para aquela rota, estourando o tempo de espera do roteiro. Não é regressão — mas é um passo que pode reprovar por tempo, e quem vier depois deve repetir a camada antes de investigar.

Ordem que o script cobra: rodar `navegador` sozinho deixa o `.next` em modo dev, e a camada `runtime` seguinte reprova com "build de dev" em 0s. O próprio script avisa e manda rodar `bash scripts/validar.sh build runtime`. Não é erro do código.

3. O que NÃO foi validado — pendências nominais

1. **Decisão 6 (`PROPOSTA_REGISTRADA`) continua aberta.** Reconfirmado no banco hoje: o `documentos_processamento_status_check` aceita só `PENDENTE`, `ERRO_VALIDACAO`, `REVISAO_HUMANA` e `APROVADO` — o valor que o workflow 4b escreve **falha**. A migration **não mexeu no CHECK**, de propósito: a decisão é espelhar o que o `d0t8aiCX20Cr08RH` escreve ao registrar contrato, e isso exige o n8n, que só carrega na próxima sessão.
2. **O bloco de verificação de 5 checagens não disparou** — os nomes estavam certos. Ele nunca foi exercitado no caminho de falha.
3. **21 dos 29 documentos ficaram sem `canal_chat_id`** (1.2). Não é pendência de trabalho, é limitação do dado histórico, mas está listada para não ser descoberta de novo.
4. **Conferência visual não foi feita.** A Fase 1 não tem tela; a camada navegador cobre o render das telas existentes e deixou screenshots em `/tmp/gc-validacao/shots`, mas ninguém olhou uma tela nova, porque não há.
5. **`contatos_whatsapp` continua com o nome antigo.** Renomear para `contatos_canal` é item opcional da Fase 8, depois do corte — renomear agora quebraria os 4 nós n8n que a consultam.

4. Observações que apareceram e não viraram trabalho

- **`documentos_processamento.obra_id` tem FK simples**, o mesmo problema corrigido em `contatos_whatsapp`. Não foi mexido: fora do escopo acordado, e alterar FK de coluna com dado é risco sem pedido. Candidato registrado.
- **`contrato_criado_id` também é FK simples.** O `proposta_criada_id` novo nasceu composto; os irmãos ficam diferentes até alguém uniformizar.
- **Profile de serviço é um só, numa empresa só** (`LC EMPRESA`). `profiles.empresa_id` é `not null` e `auth.users.email` é único. Num banco multi-empresa, usuários das outras empresas veriam "usuário removido" no histórico das propostas da automação. Hoje é teórico; a correção seria um profile por empresa.

5. Registro para quem for operar gc-prod

A `20260921150000_contatos_canal.sql` não foi aplicada em prod e não é tarefa desta frente. gc-prod segue `INACTIVE`. Quando despausar:

1. O `supabase migration repair --status applied` das dez versões **continua pendente em prod** — ver `supabase/README.md`. É pré-requisito de qualquer `db push` lá.
2. Ordem: todas as migrations até `20260908120000_orcamentos_historico.sql` primeiro; esta é a 14ª.
3. Backup antes. O backfill toca todas as linhas de `documentos_processamento`, e a troca de FK em `contatos_whatsapp` **aborta se houver contato apontando para obra de outra empresa** — em prod isso é possível, porque nada impedia até agora. O `begin/commit` faz a migration inteira voltar atrás, sem estrago.
4. `criar_usuario_automacao.sql` também precisa rodar em prod, senão a ingestão não tem autor.

6. Próximo passo

Fase 1 fechada. A rodada segue em paralelo com a sprint 5 (Itens), que é a cadência definida.

- **Numa sessão nova** (para o `n8n-mcp` carregar): Fase 0 — os dois bots, o grupo de admin, a Credential — e fechar a decisão 6 abrindo o `d0t8aiCX20Cr08RH`.
- **Ponto de contato com a sprint 5:** os helpers de item em `src/lib/`. Não existe `src/lib/itens.ts` ainda; quem chegar primeiro define o formato.